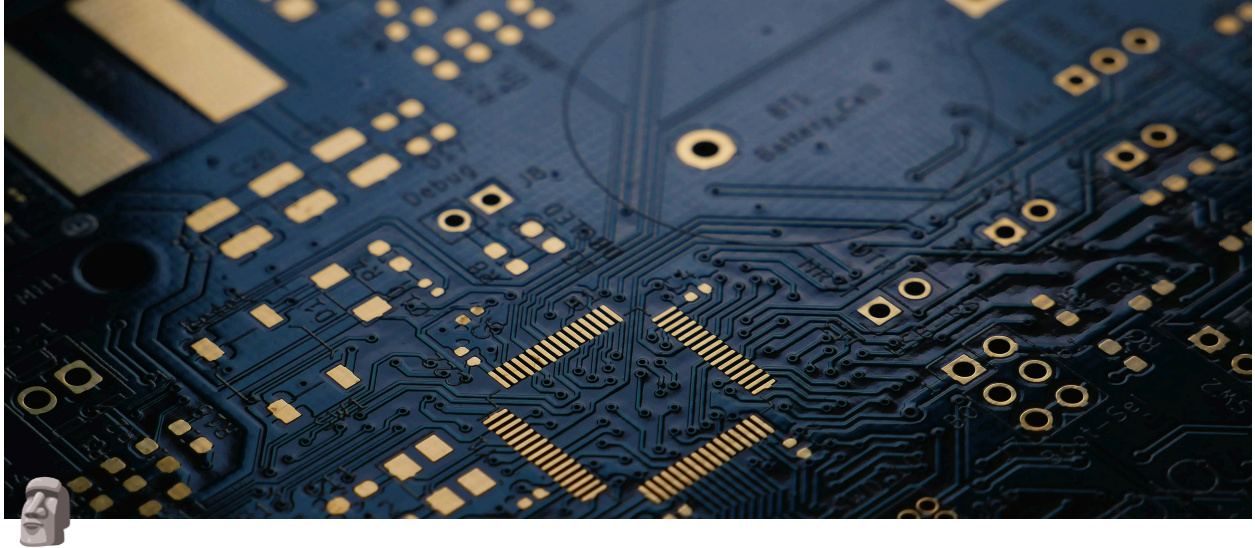
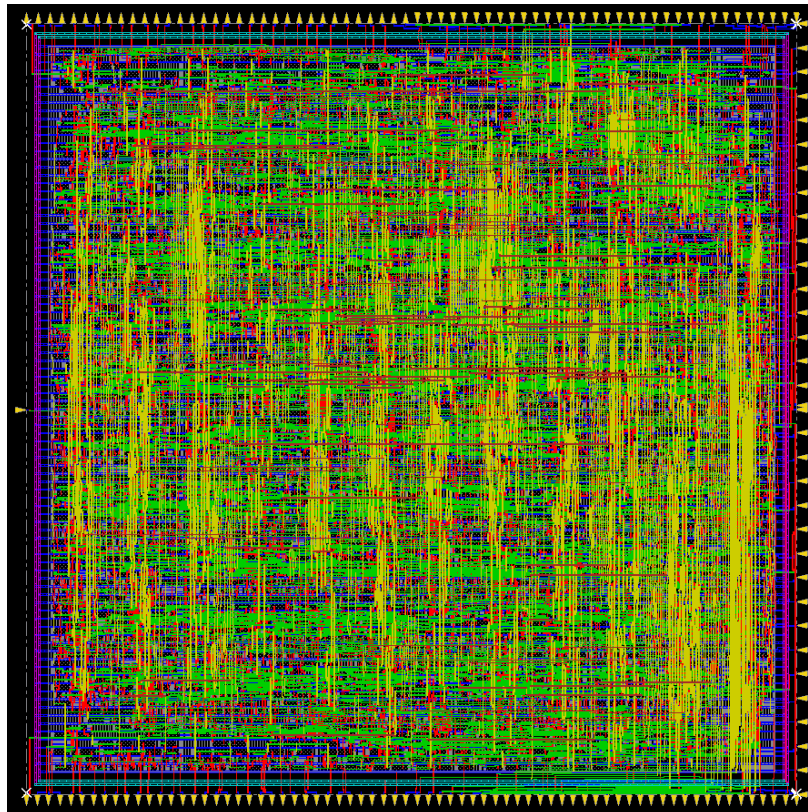


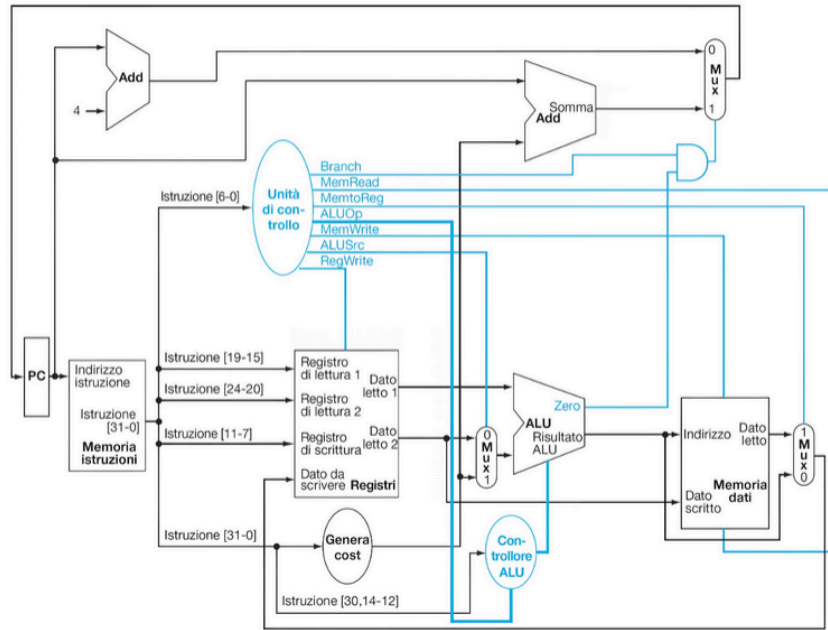


Appunti per tesi [Risc-V] (in corso)

Capitolo 1. Introduzione.



(Architettura RISK-V) fa PAURA, letteralmente.



Architettura schematizzata Pontarelli mod, fa già meno paura no?

Da sempre l'uomo ha la voglia di automatizzare. semplificare delle situazione, lo fa dall'alba dei tempi. Il primo modello di "computer" ci arriva dai Maya che provarono a predire le eclissi con dei sistemi molto avanzati se contestualizzati al periodo...

Tornando ai giorni nostri (più o meno) negli anni '30 iniziarono a nascere i primi calcolatori per come li conosciamo noi erano a valvole e semimeccanici e potevano eseguire specifiche operazioni, non erano programmabili. Se ci pensate era un enorme spreco, come se oggi inventassero un pc che fa solo una cosa.

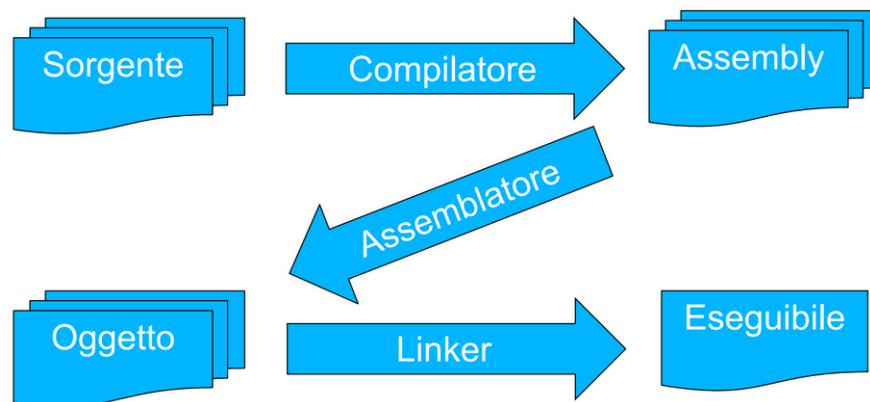
I primi pc programmabili arrivano a fine anni '40, prima programmabili con schede forate, poi programmabili in binario, MA CHE FATICA! Per questo iniziarono a nascere i linguaggi di programmazione dei linguaggi comodi agli umani e alle macchine in grado di semplificare il lavoro degli schiavi programmatori che poveretti per scrivere un ciclo for sulle schede forate non vedevano i figli per 9 giorni.

Compilatore/Assemblatore

Un compilatore è un software che traduce quello che l'uomo scrive in linguaggio leggibile dall'uomo in codice Assembly.

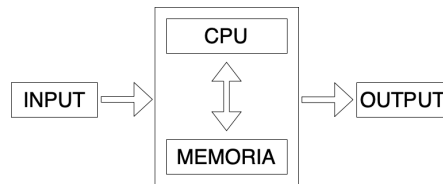
Un assemblatore è un altro software che traduce il codice assembly in codice macchina binario

01011010001010101000101001010...



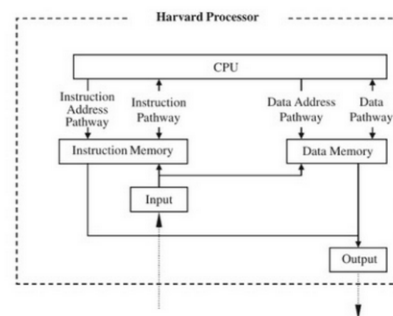
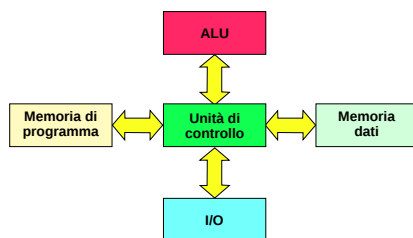
Macchina Teorica di Neumann

Neumann teorizzò che la macchina perfetta dovesse avere la seguente struttura:



tuttora i moderni calcolatori usano questa struttura di base.

Architettura Harvard



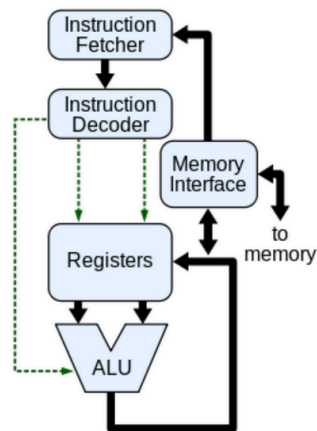
Un computer è diviso in:

- Memoria distinta per:
 - dati
 - programmi (istruzioni)
- CPU (CU + ALU + registri)
- Bus di comunicazione tra le diverse parti
Bus separato per le diverse memorie
- Periferiche di I/O
(tastiera, schermo, stampante, scheda di rete eccetera)

La CPU

La Central Processing Unit è formata da:

- La Unità di elaborazione Aritmetico/Logica (ALU)
fa solo calcoli
NON ha uno stato interno
- I registri
A uso generale e speciale,
per manipolare istruzioni, indirizzi, dati, risultati
- Il bus di comunicazione con la Memoria
- Il bus di comunicazione con le periferiche



Risc-V

Le architetture sono di due tipi principalmente, risc e cisc.

Risc = Reduced instruction set computer.

Cisc = Complex instruction set computer.

le principali differenze sono che:

nelle architetture risc le istruzioni hanno lunghezza fissa, le operazioni non attingono dati dalla memoria ma solo dai registri, c'è un modo di indirizzare la memorie semplice e di conseguenza la pipeline è più veloce.

Importante:

- Memoria RISC-V è indicizzata al byte
- Se sono all'indirizzo t e devo leggere la parola successiva incremento indirizzo come $t+4$, questo perché una parola sono 4 byte ossia 32 bit (1 byte = 8 bit)
- Il RISC-V non ha il vincolo di allineamento: le parole (DATI) non devono iniziare ad indirizzi multipli di 4 (ma per motivi di prestazione e atomicità delle istruzioni si preferisce allineare)
- Gli indirizzi sono little-endian (byte meno significativo è memorizzato per primo, nella locazione di memoria con indirizzo minore)

Comandi Risc-V

Linguaggio assembler RISC-V

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Aritmetiche	Somma	add x5, x6, x7	$x5 = x6 + x7$	Operandi in tre registri
	Sottrazione	sub x5, x6, x7	$x5 = x6 - x7$	Operandi in tre registri
	Somma immediata	addi x5, x6, 20	$x5 = x6 + 20$	Utilizzata per sommare delle costanti
Trasferimento dati	Lettura parola	lw x5, 40(x6)	$x5 = \text{Memoria}[x6 + 40]$	Spostamento di una parola da memoria a registro
	Lettura parola, senza segno	lwu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una parola senza segno da memoria a registro
	Memorizzazione parola	sw x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una parola da registro a memoria
	Lettura mezza parola	lh x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola da memoria a registro
	Lettura mezza parola, senza segno	lhu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di una mezza parola senza segno da memoria a registro
	Memorizzazione mezza parola	sh x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di una mezza parola da registro a memoria
	Lettura byte	lb x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte da memoria a registro
	Lettura byte, senza segno	lbu x5, 40(x6)	$x5 = \text{Memoria}[x6+40]$	Spostamento di un byte senza segno da memoria a registro
	Memorizzazione byte	sb x5, 40(x6)	$\text{Memoria}[x6+40] = x5$	Spostamento di un byte da registro a memoria
	Lettura di una parola e blocco	lr.d x5, (x6)	$x5 = \text{Memoria}[x6]$	Caricamento di una parola come prima fase di un'operazione atomica sulla memoria
	Memorizzazione condizionata di una parola	sc.d x7, x5, (x6)	$\text{Memoria}[x6] = x5; x7 = 0/1$	Memorizzazione di una parola come seconda fase di un'operazione atomica sulla memoria
	Caricamento costante nella mezza parola superiore	lui x5, 0x12345	$x5 = 0x12345000$	Caricamento di una costante su 20 bit nei 12 bit più significativi di una parola
Logiche	And	and x5, x6, x7	$x5 = x6 \& x7$	Operandi in tre registri; AND bit a bit
	Or inclusivo	or x5, x6, x8	$x5 = x6 x8$	Operandi in tre registri; OR bit a bit
	Or esclusivo (Xor)	xor x5, x6, x9	$x5 = x6 \wedge x9$	Operandi in tre registri; XOR bit a bit
	And immediato	andi x5, x6, 20	$x5 = x6 \& 20$	AND bit a bit tra un operando in registro e una costante
	Or esclusivo immediato	ori x5, x6, 20	$x5 = x6 20$	OR bit a bit tra un operando in registro e una costante
	Xor immediato	xori x5, x6, 20	$x5 = x6 \wedge 20$	XOR bit a bit tra un operando in registro e una costante
Scorrimento (shift)	Scorrimento logico a sinistra	sll x5, x6, x7	$x5 = x6 \ll x7$	Scorrimento a sinistra mediante registro
	Scorrimento logico a destra	srl x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra mediante registro
	Scorrimento aritmetico a destra	sra x5, x6, x7	$x5 = x6 \gg x7$	Scorrimento a destra aritmetico mediante registro
	Scorrimento logico a sinistra immediato	slli x5, x6, 3	$x5 = x6 \ll 3$	Scorrimento a sinistra mediante costante
	Scorrimento logico a destra immediato	srl i x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra mediante costante
	Scorrimento aritmetico a destra immediato	srai x5, x6, 3	$x5 = x6 \gg 3$	Scorrimento a destra aritmetico mediante costante

Salti condizionati	Salta se uguale	beq x5, x6, 100	Se (x5 == x6) vai a PC+100	Test di uguaglianza; salto relativo al PC
	Salta se non è uguale	bne x5, x6, 100	Se (x5 != x6) vai a PC+100	Test di disuguaglianza; salto relativo al PC
	Salta se minore di	blt x5, x6, 100	Se (x5 < x6) vai a PC+100	Comparazione di minoranza; salto relativo al PC
	Salta se maggiore o uguale di	bge x5, x6, 100	Se (x5 >= x6) vai a PC+100	Comparazione di maggioranza o uguaglianza; salto relativo al PC
	Salta se minore di, senza segno	bltu x5, x6, 100	Se (x5 < x6) vai a PC+100	Comparazione di minoranza senza segno; salto relativo al PC
	Salta se maggiore o uguale di, senza segno	bgeu x5, x6, 100	Se (x5 >= x6) vai a PC+100	Comparazione di maggioranza o uguaglianza senza segno; salto relativo al PC
Salti incondizionati	Salta e collega	jal x1, 100	x1 = PC+4; vai a PC+100	Chiamata a procedura con indirizzamento relativo al PC
	Salta e collega mediante registro	jalr x1, 100(x5)	x1 = PC+4; vai a x5+100	Ritorno da procedura; chiamata indiretta

Formato delle istruzioni RISC-V

Istruzioni della CPU tutte da 32 bit con formato molto simile

R-type: (tipo a Registro)

- SENZA accesso alla memoria
- Istruzioni Aritmetico/Logiche



Esempio

add x9, x20, x21

0	21	20	0	9	51
0000000	10101	10100	000	01001	0110011

somma

opcode per operazioni aritmetiche

I-type: (tipo Immediato)

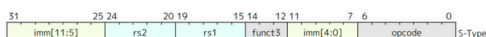
- Load
- Somma immediata



Esempio: se voglio sommare al valore in un registro un intero...

S-type: (tipo Store)

- Salvataggio registro in memoria



Capitolo 2. Iniziare a programmare in assembly RISC-V.

Principi di progettazione

1. La semplicità favorisce la regolarità
2. Minori sono le dimensioni, maggiore è la velocità
3. Un buon progetto richiede buoni compromessi

1. La semplicità favorisce la regolarità

Questo principio suggerisce che mantenere le istruzioni uniformi permette di costruire hardware più semplice e veloce. Se ogni operazione segue lo stesso schema, il processore non deve "indovinare" cosa fare.

- **Esempio pratico:** In RISC-V, le istruzioni aritmetiche hanno quasi sempre **tre operandi** (due sorgenti e una destinazione).
 - `add x1, x2, x3` (Somma x2 e x3, salva in x1)
- **Perché è importante:** Se tutte le istruzioni hanno la stessa lunghezza (32 bit) e la stessa struttura, l'unità di controllo del processore può decodificarle molto più rapidamente. La varietà eccessiva complica il design e rallenta il clock del computer.

2. Minori sono le dimensioni, maggiore è la velocità

Nel mondo dell'elettronica, la distanza fisica conta. Più i dati sono vicini al "cuore" del processore, più velocemente possono essere elaborati.

- **Il ruolo dei registri:** RISC-V utilizza un numero limitato di **registri** (32 in totale, da `x0` a `x31`).
- **La velocità della luce (e del silicio):** Un numero enorme di registri renderebbe i circuiti elettricamente più distanti e complessi, aumentando il tempo di accesso. Limitando il numero di registri a 32, si garantisce che il segnale elettrico arrivi a destinazione in una frazione minima di nanosecondo.
- **Confronto:** Accedere a un registro è ordini di grandezza più veloce che accedere alla memoria RAM, che è fisicamente "lontana" e immensa.

3. Un buon progetto richiede buoni compromessi

In ingegneria non esiste la perfezione assoluta, esistono solo scelte ottimizzate. Spesso, per guadagnare in un ambito, bisogna cedere in un altro.

- **Il compromesso tra istruzioni e dati:** Abbiamo detto che le istruzioni RISC-V devono essere regolari (32 bit). Ma cosa succede se vogliamo caricare una costante numerica molto grande?
 - Se permettessimo istruzioni di lunghezza variabile (per ospitare numeri grandi), violeremmo il principio n.1 (regolarità).
 - **La soluzione RISC-V:** Si usano istruzioni multiple per comporre un numero grande, mantenendo però la dimensione dei comandi fissa.
- **Prestazioni vs. Facilità di programmazione:** È più facile per un umano scrivere una singola istruzione complessa, ma è più veloce per il processore eseguirne tre semplici. RISC-V sceglie la velocità del processore, lasciando al **compilatore** il compito di tradurre il codice umano in piccoli passi atomici.

Gestione dell'Allineamento

- **RISC-V generico:** Non impone vincoli rigidi; le word non devono necessariamente iniziare da indirizzi multipli di 4.
- **Simulatore RARS:** Richiede invece che dati e istruzioni siano allineati alla **parola** (indirizzi multipli di 4).
- **Istruzioni:** RISC-V richiede l'allineamento alla **mezza parola** (multipli di 2 byte) per supportare istruzioni di lunghezze diverse (16, 32, 64 bit).

Allineamento memoria (esempio)

Sistema 32 bit con vincolo di allineamento

```
#include <stdio.h>

struct NonAllineata {
    char c;
    int i;
};
```

7	i[3]
6	i[2]
5	i[1]
4	i[0]
3	
2	
1	
0	c

```
struct Allineata {
    int i;
    char c;
};
```

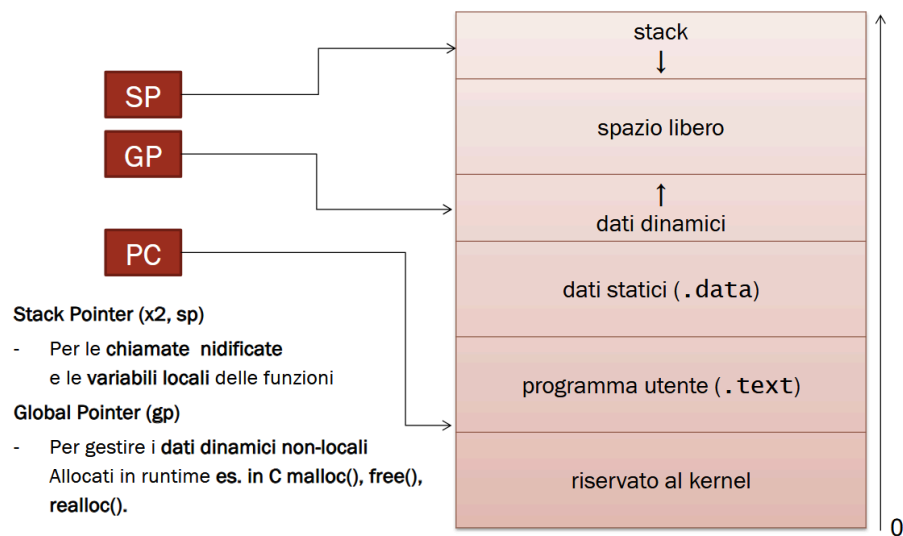
4	c
3	i[3]
2	i[2]
1	i[1]
0	i[0]

Rars richiede l'allineamento di dati e istruzioni alla parola (indirizzo in byte multiplo di 4)

Struttura della Memoria (Layout)

La memoria è organizzata in sezioni gestite da registri specifici:

- **Stack:** Gestito dallo **Stack Pointer (sp)**; usato per chiamate nidificate e variabili locali.
- **Dati Dinamici:** Gestiti dal **Global Pointer (gp)** per allocazioni runtime (es. `malloc` in C).
- **Dati Statici:** Sezione `.data`.
- **Programma Utente:** Sezione `.text`, indicizzata dal **Program Counter (PC)**.
- **Riservato al Kernel:** La porzione più bassa della memoria.



Il principio "Load/Store"

A differenza di altre architetture (come x86), in RISC-V **le operazioni aritmetiche non avvengono mai direttamente in memoria**.

- Per elaborare un dato, questo deve essere prima copiato dalla memoria a un registro (istruzione **Load**, es. `lw` per una word).
- Una volta elaborato, il risultato viene riscritto dalla CPU nella memoria (istruzione **Store**, es. `sw`).

Indirizzamento e Allineamento

- **Allineamento alla Parola (Word):** Nel simulatore RARS, la memoria è organizzata in parole da 32 bit (4 byte). Gli indirizzi delle word devono essere **multipli di 4**. Un errore di "Load address not aligned" avviene se si prova a leggere una word da un indirizzo non divisibile per 4 (es. `0x10010001`).
- RISC-V è tipicamente Little-Endian (il byte meno significativo ha l'indirizzo più basso).

Modalità di Indirizzamento dei Dati

Il processore calcola dove trovare i dati in tre modi principali:

1. **Immediato:** Il valore è scritto dentro l'istruzione stessa (es. `addi x5, x6, 10`).
2. **A Registro:** Il dato è già in uno dei 32 registri.
3. **Base + Offset:** L'indirizzo finale è la somma del contenuto di un registro base e di una costante (es. `lw t0, 4(s1)` carica il dato dall'indirizzo contenuto in `s1` + 4 byte).

Il Set di Istruzioni RISC-V

L'esecuzione di un'istruzione segue le fasi di **Fetch** (caricamento dal PC), **Decodifica** (attivazione della Control Unit), **Esecuzione** (tramite ALU), **Store** (salvataggio dei risultati) e **Aggiornamento del PC**.

Tipologie di Istruzioni

- **LOAD/STORE:** Trasferimento dati tra memoria e registri (es. `lw`, `sw`, `lb`).
- **Logico-Aritmetiche:** Calcoli interni (es. `add`, `sub`, `and`, `or`, `sll`).
- **Salti:** Controllo del flusso (es. `beq`, `blt`, `jal`).
- **Pseudoistruzioni:** Istruzioni mnemoniche tradotte dall'assemblatore in istruzioni hardware reali (es. `mv x10, x11` diventa `addi x10, x11, 0`).

Il set di istruzioni (cosa la CPU «sa fare»)

Le fasi di esecuzione di una istruzione sono:

- **Fetch**/caricamento della istruzione
Dalla posizione indicata dal Program Counter
- **Decodifica**/riconoscimento della istruzione.
La Control Unit attiva le parti funzionali necessarie
- **Load**/caricamento di eventuali **argomenti**
A seconda dei modi di indirizzamento (vedi dopo)
- **Esecuzione** della istruzione
In genere da parte dell'ALU
- **Store**/salvataggio del risultato
In memoria o in un registro
- **Aggiornamento del Program Counter**
(contemporaneamente ad altre fasi)

Tipologie di istruzioni:

- **LOAD/STORE**
Trasferiscono dati da/verso la memoria
Es: lb, lbu, lh, lhu, lui, lw, sb, sh
- **Logico/Aritmetiche**
Svolgono i calcoli aritmetici e logici
Es: add, and, sub, subw, or, sll, ...
- **Salti** condizionati e incondizionati
Controllano il flusso logico di esecuzione
Es: beq, bge, bgeu, blt, jal, ...
- **Gestione delle eccezioni/interrupt**
Salvataggio dello stato e suo ripristino
Es: xRET
- Istruzioni di **trasferimento dati**
Non necessarie col memory-mapping

Dagli appunti del corso

Salti Condizionati (Branches)

Questi salti avvengono **solo se una determinata condizione è vera**. Se la condizione è falsa, il processore ignora il salto e continua con l'istruzione successiva (quella subito sotto).

- **Istruzioni tipiche:** `beq` (branch if equal), `bne` (not equal), `blt` (less than), `bge` (greater or equal).
- **Logica:** Confrontano due registri.
- **Esempio:** "Se il registro `x1` è uguale a `x2`, vai all'etichetta `L1`. Altrimenti, prosegui."
- **Range:** Hanno solitamente un raggio d'azione limitato perché l'offset (la distanza del salto) deve entrare in una parte dell'istruzione a 32-bit.

Salti Incondizionati (Jumps)

Questi salti avvengono **sempre**. Non c'è alcun controllo o confronto; il "PC" (Program Counter) viene forzato a puntare a un nuovo indirizzo.

- **Istruzioni tipiche:** `j` (pseudo-istruzione per `jal`), `jal` (jump and link), `jalr` (jump and link register).
- **Logica:** Servono per chiamate a funzioni o salti diretti a una parte diversa del codice.
- **Il concetto di "Link":** Spesso questi salti salvano l'indirizzo dell'istruzione successiva in un registro (di solito `ra`, ovvero *return address*), così il programma sa dove tornare dopo aver finito una funzione.
- **Range:** Il `jal` può coprire una distanza molto più ampia rispetto ai branch.

Formati delle Istruzioni (32-bit)

Le istruzioni hanno lunghezza fissa per semplificare la pipeline:

- **R-type:** Operazioni tra registri, senza accesso alla memoria.
- **I-type:** Operazioni con valori immediati (costanti) o carichi dalla memoria.
- **S-type:** Operazioni di memorizzazione (Store).
- **U-type:** Per immediati grandi (20 bit), usati per costruire costanti a 32 bit.
- **UJ-type/SB-type:** Varianti per salti e rami condizionati relativi al PC.

Registri della CPU

RISC-V dispone di **32 registri ad uso generale** (da x0 a x31).

Registro	Nome	Utilizzo principale
x0	zero	Costante fissa 0
x1	ra	Indirizzo di ritorno (Return Address)
x2	sp	Stack Pointer
x3	gp	Global Pointer
x5-x7	t0-t2	Variabili temporanee (non preserve)
x10-x17	a0-a7	Argomenti di funzioni e risultati
x18-x27	s2-s11	Registri salvati (preservati dal chiamato)

in realtà dal 5 al 27 possiamo usarli liberamente come vogliamo, la separazione è solo concettuale.

Modalità di Indirizzamento

Il documento identifica quattro modalità principali in RISC-V:

1. **Immediato:** L'operando è contenuto nell'istruzione stessa.
2. **A Registro:** L'operando si trova in un registro.
3. **Base + Spiazzamento (Offset):** L'indirizzo di memoria è calcolato sommando una costante al contenuto di un registro.
4. **Relativo al PC:** L'indirizzo è calcolato sommando un immediato al Program Counter (usato per i salti).

Strumenti e Direttive Assembler

Il software di riferimento è **RARS** (RISC-V Assembler and Runtime Simulator). Per scrivere programmi corretti si usano direttive come:

- `.data` e `.text` per separare dati e codice.
- `.word`, `.half`, `.byte` per allocare spazio dati.
- `.asciz` per le stringhe terminate da carattere nullo.
- **Etichette:** Usate per evitare il calcolo manuale degli indirizzi di salto.

Assembly RISC-V

Direttive principali per l'assemblatore

.data	definizione dei dati statici	Codifica mnemonica dei registri
.text	definizione del programma	a0, sp, ra ... f0, f31
.asciz	stringa terminata da zero	Etichette (per calcolare gli indirizzi relativi) nome:
.byte	sequenza di byte	L'assemblatore converte
.double	sequenza di double	-dal testo del programma in assembly
.float	sequenza di float	-il programma in codice macchina
.half	sequenza di half words	
.word	sequenza di words	
.globl sym	dichiara il simbolo come globale e può essere referenziato da altri file	-Dalle etichette calcola gli indirizzi Dei salti relativi Delle strutture dati in memoria (offset) Dei salti assoluti

Codici mnemonici delle **istruzioni**

add, sub, beq ...

Pseudoistruzioni: istruzioni non implementati in hardware, ma che possono essere tradotte in istruzioni macchina.

Es. `mv x10, x11` → `addi x10, x11, 0`

NOTA: le strutture di controllo del flusso del programma vanno realizzate «a mano» usando i salti condizionati e le etichette

dalle diapositive del corso

Traduciamo programmi basilari C in Assembly RISC-V

Prendere decisioni (if then else)

```
if (i == j) f = g + h; else f = g - h;
```

```
bne s3,s4,Else      # vai a Else se i ≠ j
add s0,s1,s2         # f = g + h (saltata se i ≠ j)
j Esci               # vai a Esci
Else: sub s0,s1,s2    # f = g - h (saltata se i = j)
Esci:
```

bne = branch if not equal

in s1, s2, s3 e s4 salvo i dati e in s0 salvo il risultato della somma

"Else" ed "Esci" sono etichette.

Altri esempi:

Realizzare Iterazioni (ciclo do while)

Esempio C

```
do {  
  
    // codice da ripetere se x != 0  
    // il corpo del ciclo DEVE aggiornare x  
  
} while (x != 0);  
  
// codice seguente
```

Esempio Assembly

```
.text  
# uso il registro t0 per l'indice x  
  
do:  
  
# codice da ripetere  
  
bnez t0, do    # test x != 0  
  
# codice seguente
```

bnez = brench if not equal zero

Realizzare Iterazioni (while do)

Esempio C

```
while (x != 0) {  
  
    // codice da ripetere se x != 0  
    // il corpo del ciclo DEVE aggiornare x  
}  
  
// codice seguente
```

Esempio Assembly

```
.text  
# uso il registro t0 per l'indice x  
  
while:  
beqz t0, endWhile    # test x == 0  
  
# codice da ripetere  
  
j while              # loop  
  
endWhile:  
# codice seguente
```

Realizzare Iterazioni (for loop)

Esempio C

```
for (i=0 ; i<N ; i++)
{
    // codice da ripetere
}

// codice seguente
```

Esempio Assembly

```
.text
    # uso il registro t0 per l'indice i
    # uso il registro t1 per il limite N
    xor t0, t0, t0          # azzero i
    li t1, N                # limite del ciclo

    cicloFor:
        bge t0, t1, endFor   # test i >= N
        # codice da ripetere
        addi t0, t0, 1        # incremento di i
        j cicloFor           # loop

    endFor:
        # codice seguente
```

SWITCH CASE

Esempio C

```
switch (A) {
case 0: // codice del caso 0
    break;
case 1: // codice del caso 1
    break;
// altri casi
case N: // codice del caso 3
    break;
}

// codice seguente
```

Esempio Assembly

```
.text
    lbu t0, A               # selezioniamo caso A
    slli t0, t0, 2          # A*4
    la t1, dest             # indirizzo primo case
    add t1, t0, t1          # indirizzo selezionato
    lw t1, (t1)             # carico indirizzo
    jr t1                   # salto a registro

    caso0:                  # codice del caso 0
        j endSwitch
    caso1:                  # codice del caso 1
        j endSwitch
    casoN:                  # codice del caso N
        j endSwitch
    endSwitch:              # codice seguente

.data
    dest: .word caso0, caso1, ..., casoN
    A:    .byte
```

Pseudo-istruzioni

Ci sono delle operazioni che sono più operazioni in serie ad esempio:

Il blez (branch if less than or equal to zero) è fittizio. E' in realtà stato implementato con bge.

La direttiva globl

La useremo ma non è essenziale quando si lavora con un solo file come nel nostro caso.

E' utile quando abbiamo più file da gestire e abbiamo delle parti del codice che referenziano etichette che sono in file diversi.

Primo esempio di programma

Es.: trova il max di un vettore

Esempio C	Esempio Assembly
<pre>// definizione dei dati int vettore[6] = { 11, 35, 2, 17, 29, 95 }; int N = 6; int max = vettore[0]; // scansione il vettore for (i=1 ; i<N ; i++) { int el = vettore[i]; // el. corrente if (elemento > max) max = elemento; }</pre>	<pre>.data vettore: .word 11, 35, 2, 17, 29, 95 N: .word 6 .text la s0, vettore lw t0, 0(s0) # max → \$t0 lw t1, N # N → \$t1 li t2, 1 # i = 1 for: bge t2, t1, endFor slli t3, t2, 2 # i*4 add t3, t3, s0 # t3 = vettore+i*4 lw t4, (t3) # el. = vettore[i] ble t4, t0, else # if (el >= max) mv t0, t4 # max = el. else: addi t2, t2, 1 # i++ j for endFor:</pre>

Capitolo 3. Strutture dati: Vettori e Matrici.

Un **vettore** è definito come una sequenza di N elementi di uguali dimensioni, memorizzati in modo consecutivo.

- **Indirizzamento:** Gli elementi sono numerati da 0 a N-1.
- **Calcolo dell'Offset:** Per accedere all'elemento i-esimo, bisogna sommare all'indirizzo base del vettore uno spostamento (offset) pari a $i \times \text{dimensione_elemento}$ (del vettore).
- **Definizione in Assembly:** Si dichiarano nella sezione `.data` usando etichette e direttive come `.word` (4 byte), `.half` (2 byte) o `.byte` (1 byte).

I caratteri ASCII vengono memorizzati utilizzando la direttiva `.asciz` o `.byte`.

- **.asciz:** Questa direttiva definisce una stringa di caratteri seguita automaticamente da un carattere **null** (valore 0), che funge da terminatore.
- **Dimensione:** Ogni carattere ASCII occupa esattamente **1 byte** (8 bit).

Le Matrici (Vettori di Vettori)

Le matrici sono memorizzate linearmente (una riga dopo l'altra).

- **Matrice 2D:** L'elemento $M[x][y]$ in una matrice con `NUM_COL` si trova all'indirizzo:
 $\text{base} + (x \times \text{NUM_COL} + y) \times \text{dimensione_elemento}$
- **Matrice 3D:** Per una matrice $L \times M \times N$, l'elemento (l, x, y) si trova all'offset:
 $l \times (M \times N) + x \times N + y$

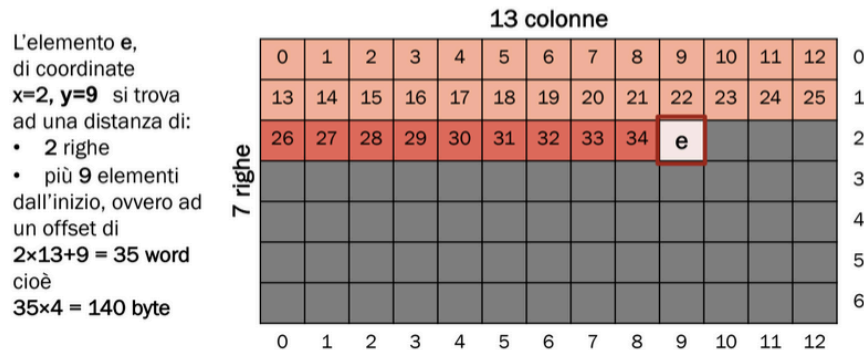
(Non si capisce un cazzos lo so) capiamolo.



Matrici: vettori di vettori

Una matrice $M \times N$ è altro una successione di M vettori, ciascuno di N elementi

- il numero di elementi totali è: $M \times N$
 - la dimensione totale in byte è: $M \times N \times \text{dimensione_elemento}$
 - la si definisce staticamente come un vettore contenente $M \times N$ elementi uguali
- Matrice:** `.word 0:91` # spazio per una matrice 7×13 word



dalle diapositive del corso.

Endianness (Ordinamento dei Byte)

- **Little-endian:** Il byte meno significativo (LSB) è memorizzato all'indirizzo più basso. È il sistema usato da **Intel** e **RARS**.
 - *Esempio:* La word `0x01020304` viene memorizzata come `04 03 02 01`.
- **Big-endian:** Il byte più significativo (MSB) viene memorizzato per primo.
 - *Esempio:* La word `0x01020304` viene memorizzata come `01 02 03 04`.

Big-endian non ci interessa.

Gestione dei Cicli e Scansione

Esistono due approcci principali per scorrere un vettore in Assembly RISC-V:

Metodo	Pro	Contro
Scansione per Indice	Comoda per controlli logici; incremento indipendente dalla dimensione.	Richiede di calcolare l'offset in byte a ogni iterazione.
Scansione per Puntatore	Più veloce; meno calcoli all'interno del ciclo.	L'incremento dipende dalla dimensione dell'elemento; l'indice non è subito disponibile.

Esempio Pratico: Somma di elementi (Indice dispari vs pari)

Il documento propone un esercizio complesso: calcolare due somme distinte (elementi a indice pari e elementi a indice dispari).

Logica della soluzione fornita :

1. **Inizializzazione:** Si azzerano i registri per le somme (`s0` per dispari, `s3` per pari) e si carica la lunghezza del vettore `N`.
2. **Ciclo (while):** Si scorre il vettore finché l'indice `t1` è minore di `N`.
3. **Flag di Parità:** Viene usato un registro (`t3`) come "interruttore" (flag) che viene invertito a ogni iterazione per determinare se l'indice corrente è pari o dispari.
4. **Accesso alla Memoria:** Si calcola l'offset ($t1 \times 4$), si somma all'indirizzo base e si carica il valore.
5. **Accumulo:** In base al flag, il valore viene aggiunto alla somma corrispondente.

6. **Salvataggio:** Al termine, i due risultati vengono scritti in dei registri.

```
1 .data
2     vettore: .word 1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20
3     N: .word 20
4     # modifica questi valori insieme se vuoi sommare valori diversi
5 .text
6     la s0, vettore
7     lw t0, 0(s0)
8     lw t1, N
9     li t2, 1
10    li s9, 2
11    for: bgt t2, t1, endFor
12        add t3, t3, s0
13        lw t0, (t3)
14        slli t3, t2, 2 # SE HO 1 DIVENTA 4, SE HO 2 DIVENTA 8 ... SE HO 4 DIVENTA 16... MOLTIPLICAZIONE PER 4.
15        rem t4, t2, s9 # calcola resto
16        ble t4, zero, pari
17    dispari:
18        add s4, s4, t0
19        j fine
20    pari:
21        add s5, s5, t0
22        j fine
23    fine:
24        addi t2, t2, 1
25        j for
26    endFor:
27
28    # Questo programma somma i numeri in posizione pari e li colloca in s5 il risultato,
29    # somma i numeri in posizione dispari e colloca in s4 il risultato.
30    # Modifica i valori in .data se vuoi sommare numeri diversi.
31
```

mio codice.

Capitolo 4. System Calls (Chiamate di Sistema).

Le **system calls** sono richieste che il software applicativo invia al sistema operativo per interagire con l'hardware o l'ambiente di esecuzione (es. input/output).

- **Convenzioni RISC-V (RARS):**
 - **Registro a7** : Contiene il codice numerico dell'operazione richiesta.
 - **Registri a0...a6** : Contengono i parametri di input.
 - **Registro a0** : Contiene il valore restituito dal sistema.

Codice (a7)	Descrizione	Parametri principali
1	Stampa Intero	a0 = intero da stampare
4	Stampa Stringa	a0 = indirizzo della stringa
5	Leggi Intero	Risultato in a0
10	Exit	Termina il programma
1024	Apri File	a0 = path, a1 = flags

programma Hello world;

```

.globl main

.data

string: .asciz "Hello world!"

.text
main:

    li a7, 4

    la a0, string

    ecall

```

Procedure e Funzioni

Le procedure sono frammenti di codice riutilizzabili che ricevono argomenti e restituiscono risultati.

- **Istruzioni Chiave:**

- **jal rd, etichetta (Jump and Link):** Salva l'indirizzo dell'istruzione successiva (PC+4) nel registro `rd` (solitamente `ra`) e salta all'etichetta.
- **jr ra (Jump Register):** Ritorna al chiamante saltando all'indirizzo salvato in `ra`.

esempio:

```
-- PROGRAMMA PRINCIPALE (Il Chiamante) --
```

```

addi a0, zero, 5      # Carica un parametro
jal ra, raddoppia     # CHIAMATA: salva la riga successiva in 'ra' e salta
addi s1, a0, 0         # <--- IL RITORNO: il programma riprende da qui!
...

```

```
-- FUNZIONE (Il Chiamato) ---
```

```

raddoppia:
add a0, a0, a0        # Esegue il calcolo
jr ra                 # RITORNA AL CHIAMANTE: salta all'indirizzo in 'ra'

```

- **Convenzioni sui Registri:**

- `a0...a7`: Usati per passare fino a 8 argomenti e restituire i risultati.
- `t0, t1...` (**Temporary**): Registri che possono essere modificati liberamente dalla funzione chiamata.

- **s0, s1...** (**Saved**): Registri che devono mantenere il loro valore originale dopo la chiamata (se usati, la funzione deve salvarli e ripristinarli).

Preservare il contenuto dei registri

Conviene **preservare** il precedente contenuto dei registri usati dalla funzione e **ripristinarlo**

- **meno vincoli** alla funzione chiamante
- nelle funzioni che chiamano altre funzioni, che perderebbero il contenuto almeno di **ra**
- **a0, a1, ..., a7** possiamo solo passare 8 argomenti

Le informazioni da preservare hanno un **ciclo di vita caratteristico**, dovuto al **nidificarsi delle chiamate** delle funzioni:

- salvo stato prima di chiamata 1
 - salvo stato prima di chiamata 2
 - ...
 - ripristino stato prima di chiamata 2
 - ripristino stato prima di chiamata 1

Questo è il comportamento di una pila (**stack** o LIFO), in cui aggiungere un elemento (**push**) e togliere l'ultimo inserito (**pop**)

Viene realizzata con un vettore di cui si tiene l'**indirizzo dell'ultimo elemento occupato** nel registro **sp** (Stack Pointer)

dalle diapositive del corso, utilizzo di stack pointer (sp) nel capitolo 5.

Subroutines

così posso creare delle sub routines:

Case Switch Program + System Calls + Subroutines

```

.globl main
.data
    switch_case: .word case_00, case_01
    case_string: .asciz "Ran case #: "
    input_question: .asciz "Input 0 for s
    address_of_forbidden: .word forbidden
    hidden_text: .asciz "forbidden!"
.text
main:
    jal read_int # Leggi intero
    mv t2,a0 # mettilo in t2

    jal read_int # Leggi intero
    mv t3,a0 # mettilo in t3

    la a0, input_question # ask question
    jal print_string

    jal read_int # Leggi caso 0 o 1
    mv t1,a0
    slli t0, t1, 2

    la t4, switch_case
    add t4, t4, t0
    lw s1, (t4) # load the correspondin

    la a0, case_string
    jal print_string # Print fi

    mv a0,t1 # Print 0 or 1
    jal print_rez
    jr s1

case_00:
    # sum
    add t3,t3,t2
    j end_switch

case_01:

```

```

.globl print_string, read_int, print_rez, print_char, exit
.data
    # Syscall lookup table
    print_s: .byte 4
    print_i: .byte 1
    read_i: .byte 5
    print_c: .byte 11
    exit_s: .byte 10
.text
print_string:
    lb a7, print_s # cenno a una procedura
    ecall # print a string (4 means
    jr ra

print_rez:
    lb a7, prin
    ecall
    jr ra

read_int:
    lb a7, read
    ecall
    jr ra

print_char:
    lb a7, prin
    ecall
    jr ra

exit:
    lb a7, exit_s
    ecall

```

Capitolo 5. Uso della memoria.

Preservazione dei Registri e Chiamate di Funzione

Quando una funzione viene chiamata, è fondamentale preservare il contenuto dei registri per evitare conflitti tra la funzione chiamante e la funzione chiamata.

Perché preservare i registri?

- **Riduzione dei vincoli:** La funzione chiamante non deve preoccuparsi che i propri dati vengano sovrascritti.
- **Gestione dell'indirizzo di ritorno:** Nelle funzioni nidificate (funzioni che ne chiamano altre), il registro `ra` (Return Address) verrebbe sovrascritto da ogni nuova istruzione `jal`, perdendo il punto di ritorno al main.
- **Limiti dei registri:** Poiché i registri per gli argomenti (da `a0` a `a7`) sono limitati, lo stack permette di gestire una quantità maggiore di dati.

Lo Stack (Pila) e il Registro `sp`

Per gestire il salvataggio dei dati in modo ordinato, si utilizza una struttura dati di tipo **LIFO** (Last In, First Out), chiamata **Stack**.

- **Ciclo di vita:** Le informazioni salvate per prime sono le ultime a essere ripristinate (comportamento a "matrioska" dovuto alle chiamate nidificate).
- **Crescita della memoria:** Lo stack risiede nella parte alta della memoria e si espande verso il basso (indirizzi decrescenti).
- **Stack Pointer (`sp`):** È il registro che punta all'ultimo elemento occupato nello stack.

Operazioni di Push e Pop

Le operazioni fondamentali per interagire con lo stack sono l'inserimento (Push) e l'estrazione (Pop).

1. Salvataggio (Push)

Per salvare un elemento (es. il registro `ra` che occupa 4 byte):

1. **Decremento dello `sp`:** Si "crea spazio" sottraendo la dimensione del dato allo Stack Pointer.
2. **Memorizzazione (Store):** Si scrive il dato nell'indirizzo puntato da `sp`.

Esempio di codice:

Snippet di codice

```
addi sp, sp, -4 # Decrementa sp di 4 byte
sw ra, 0(sp)    # Salva il contenuto di ra nello stack
```

2. Ripristino (Pop)

Per recuperare l'elemento salvato:

1. **Caricamento (Load):** Si legge il dato dalla memoria tramite lo `sp`.
2. **Incremento dello `sp`:** Si "libera lo spazio" riportando il puntatore alla posizione precedente.

Esempio di codice:

Snippet di codice

```
lw ra, 0(sp)    # Carica il valore dallo stack nel registro ra
addi sp, sp, 4   # Incrementa sp di 4 byte per liberare lo spazio
```

Nota importante: Dopo un'operazione di "Pop", il valore in memoria non viene fisicamente cancellato; rimane lì finché non viene sovrascritto da una nuova operazione di "Push", ma non è più considerato parte dello stack attivo.

Esempio Funzioni Annidate

Uso dello stack in una funzione

funzione:

All'inizio della funzione:

- **allocare** su stack abbastanza word da contenere i registri da preservare
- **salvare** su stack i registri, ad offset multipli di 4 rispetto a `sp`

```
addi sp, sp, -12
sw ra, 8(sp)
sw s0, 4(sp)
sw s1, 0(sp)
```

NOTA: conviene allocare tutto lo spazio assieme per avere **offset che restano costanti** durante tutta l'esecuzione della funzione

```
# corpo della funzione
# posso chiamare jal
# tranquillamente perché
# salvo ra
```

All'uscita della funzione:

- **ripristinare** da stack i registri salvati, agli stessi offset usati precedentemente
- **disallocare** da stack lo stesso spazio allocato in precedenza
- **tornare** alla funzione chiamante

```
lw s1, 0(sp)
lw s0, 4(sp)
lw ra, 8(sp)
addi sp, sp, 12
jr ra
```

dalle diapositive del corso. quando facciamo `[addi sp, sp, -12]` quel `-12` indica `-12` byte.